Cognitive Merkle Trees: Persistent Cognitive State as a Versioned Intermediate Representation

Run Wang[1,*], Wei Zhou[2]
[1]BootLoader Labs [2]Baidu Inc.

Abstract

Personalized language-model systems can retain information about a person while leaving undefined the durable systems object when evidence grows, the state-building procedure changes, or the foundation model is replaced. We introduce *persistent cognitive state* (PCS): a versioned, external, evidence-addressable intermediate representation of documented cognition, distinct from both the source archive and the model that executes it. We require *person-specific state identity to factor through evidence and compilation, not through the runtime executor*. This yields a three-clock problem with independently evolving evidence time t, state-compiler version γ, and executor version τ. We introduce *Cognitive Merkle Trees* (CMT) as a reference architecture for this lifecycle. CMT combines immutable temporal artifacts, conservative relation-aware reconciliation, typed temporal-extension witnesses, support-versus-context lineage, a no-semantic-self-feed rule, revocation-local regeneration, and content-addressed commitments that bind emitted state to its derivation manifest. Hashing authenticates artifact identity and ancestry, not semantic truth. We state the resulting lifecycle laws as mechanically testable structural invariants and provide a deterministic checker with positive and adversarial fixtures. We also introduce CMT-L5, a leakage-controlled longitudinal corpus of 29,321 public authored posts from five anonymized individuals. In a bounded one-author conformance stress test, a heuristic unsupported-horizon diagnostic falls from 70 flagged items to 1; this is a contract result, not an accuracy claim. Finally, we specify a budget-matched, future-held-out behavioral evaluation that freezes candidate state before later authored evidence is opened. The broader thesis is: *evidence is memory; state is interpretation; intelligence is execution*.

Keywords: persistent cognitive state; longitudinal memory; provenance; personalization; versioned intermediate representation; agent memory.

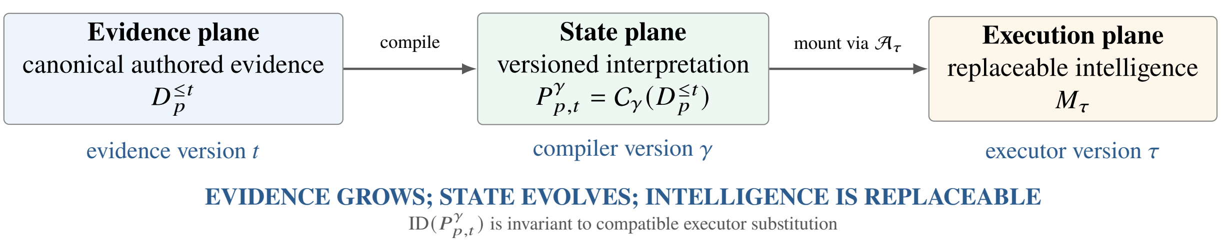

Figure 1: The three-plane separation and its identity boundary. Evidence, compiled person state, and execution are independently versioned. An executor-specific adapter $\mathcal{A}_\tau$ may select or serialize state at runtime, but neither the adapter nor M_τ belongs to persistent state identity. Output behavior may change across executors even when state identity does not.

*Corresponding author: run@bll.ac

1 Introduction

Foundation models are becoming more capable while remaining fundamentally impersonal. A model can be conditioned on a writing sample, connected to retrieval, given a running summary, or adapted to user data. These techniques are useful, but they leave a basic systems question under-specified: *what is the durable person-specific object when evidence grows, the state-building procedure improves, and the foundation model itself is replaced?*

We call that object *persistent cognitive state* (PCS). The word *cognitive* is used here as an engineering term scoped to *documented cognition*: patterns supported by authored evidence, including expressed knowledge, stance, preference, reasoning procedure, decision tendency, uncertainty, and language. We make no claim to recover unexpressed mental state, consciousness, or philosophical identity.

Research-program terminology. In the broader BootLoader Labs research program, a portable person-specific representation of this kind is called a *Human Image*. We use PCS throughout the paper as the formal systems term: *Human Image* names the portable research object, while PCS specifies the version, provenance, temporal, and executor-independence semantics required of that object. Under the same terminology, CMT is a reference architecture for compiling longitudinal authored evidence into a versioned, auditable Human Image; a compatible foundation model plus its runtime adapter forms the execution layer referred to as a *Human Emulator*. The broader program term *Human Continuity* concerns repeated synchronization between an evolving biological individual and successive digital-state versions. That continuity objective motivates the architecture but is not an empirical or philosophical claim established by this paper.

Let $D_p^{\le t}$ denote evidence authored by person p up to evidence time t. Let γ identify a state-compiler version and M_τ a foundation model available at model time τ. We write

$$P_{p,t}^{\gamma} = C_\gamma(D_p^{\le t}), \qquad \widehat{y} = M_\tau\Big(q \oplus \mathcal{A}_\tau(P_{p,t}^{\gamma}; q)\Big), \tag{1.1}$$

where $\oplus$ denotes runtime mounting, not parameter composition, and $\mathcal{A}_\tau(\cdot; q)$ is a transient, possibly query-dependent executor adapter. Equation (1.1) is the paper's central separation: evidence is canonical memory, state is a versioned interpretation of that evidence, and the model is an execution substrate. The adapter may select, order, or serialize state for a compatible executor, but it may not mint new person-specific evidence or redefine the persistent state identity.

The distinction matters because the three layers fail differently. Long context provides capacity but not a persistence lifecycle, and effective use can degrade with position and task structure even within nominal windows [1, 2, 3]. Retrieval-augmented generation provides access to evidence but does not by itself specify whether a prior stance is current, historical, superseded, or unresolved [4]. Recurrent summaries are compact but can recursively turn previous interpretation into pseudo-evidence. Person-specific parameter adaptation couples representation to a model family and weakens evidence-level audit. Long-term memory systems increasingly address retention, indexing, temporal organization, and transfer [5, 6, 7, 8]; person-centric systems increasingly address dynamic profiles and structured person understanding [9, 10, 11]. Provenance-oriented systems further show the value of versioning, cryptographic lineage, and portable memory [12, 13, 14]. What remains insufficiently isolated is the lifecycle of an independently versioned *person state* that survives both compiler and executor replacement.

Central thesis. Our claim is not that memory should be a tree. It is that *person-specific state should be a versioned intermediate representation* between authored evidence and replaceable intelligence. A concrete state artifact may change when evidence or compilation changes; what persists is an externally governed *state lineage* whose versions remain attributable, reproducible at the contract level, and independent of executor replacement.

CMT is a reference architecture for that lifecycle. Historical temporal artifacts are immutable once committed. Higher-order state reconciles child states conservatively rather than assuming semantic reduction is associative or uniquely correct. Temporal persistence must be explicit rather than silently inferred from absence of later evidence. Every derived item retains an exact support path and attribution role. Artifact hashes commit the emitted payload and its derivation manifest. A new root is regenerated from evidence-bearing descendants, never from the previous root as semantic evidence. The result behaves more like a versioned materialized view over an event history [15] than an ever-rewritten profile.

Contributions. This paper makes six contributions.

- **Persistent cognitive state and its identity invariant.** We define the durable person-specific object as an independently versioned state lineage between evidence and execution, with identity determined by evidence and compilation rather than runtime model weights.
- **The three-clock problem.** We distinguish evidence time t, compiler version γ, and executor version τ, making evidence evolution, reinterpretation, and executor improvement separately measurable interventions.

- **Cognitive Merkle Trees.** We give a reference architecture with immutable temporal artifacts, conservative relation-aware reduction, typed temporal-extension witnesses, support-versus-context lineage, no semantic self-feed, and content-addressed derivation identities.
- **Lifecycle laws and an executable conformance surface.** We formalize evidence closure, temporal non-retroactivity, attribution separation, non-recurrent regeneration, revocation locality, and executor-substitution invariance, and ship a deterministic checker with positive and adversarial fixtures. These are structural contracts, not claims of semantic infallibility.
- **A longitudinal evaluation substrate plus a bounded contract test.** CMT-L5 contains 29,321 public authored posts from five anonymized individuals with strict 70/80/90% chronological cutoffs; a one-author stress test reduces heuristic unsupported-horizon flags from 70 to 1 under the temporal contract.
- **A falsifiable behavioral protocol.** We specify a future-held-out, budget-matched evaluation that freezes candidate state before later evidence is opened and explicitly separates behavioral hypotheses from conformance results.

Why a new systems object is needed. The claim is not that existing memory systems cannot persist information. The stronger point is that persistence alone does not define the lifecycle of a *derived person interpretation*. A retrieval index can persist while leaving temporal status unresolved; a mutable profile can persist while erasing its own derivation history; and a parameter delta can persist while remaining tied to one executor family. PCS isolates the object whose identity, temporal semantics, and provenance survive those changes.

Abstraction	Persists	Temporal status	Item lineage	Root replay	Executor-neutral	What remains under-specified by itself
Long context	no	implicit	raw	n/a	yes	lifecycle beyond one mounted window
Retrieval index	yes	implicit	raw	n/a	yes	coherent lifetime interpretation
Mutable profile	yes	weak	coarse	often recurrent	yes	immutable history and exact derivation
Model-bound adaptation	yes	implicit	weak	n/a	no	cross-executor state identity
PCS / CMT	**yes**	**explicit**	**transitive**	**non-recurrent**	**yes**	**semantic compiler accuracy remains empirical**

Table 1: The object-boundary argument. These rows describe what each abstraction defines *by itself*, not the maximum capabilities of systems that combine several abstractions.

	Claimed by CMT	Not claimed
Object boundary	Documented person-specific state can be externalized as an independently versioned artifact.	The artifact is a complete human mind or private internal state.
Temporal semantics	Historical, current, superseded, and unresolved claims can coexist without rewriting frozen history.	Silence automatically confirms or invalidates a prior claim.
Merkle layer	Hash commitments authenticate artifact identity, manifest, and derivation ancestry.	Cryptographic commitment makes generated semantics true.
Portability	The same state identity can be executed by compatible models without person-specific weight retraining.	Different backbones or adapters must behave identically.
Evaluation	Future-authored evidence permits falsifiable tests of person-state representations.	Behavioral similarity proves consciousness or identity continuity.

Table 2: Claims and non-claims. CMT is an architecture for persistent, auditable representations of documented cognition, not a claim about subjective identity.

2 Persistent Cognitive State

2.1 Evidence, state, and execution are different objects

For person p, let the authored evidence stream be

$$D_p = \{e_i = (\iota_i, t_i, b_i, m_i)\}_{i=1}^{n}, \qquad t_i \leq t_{i+1}, \tag{2.1}$$

where ι_i is a stable evidence identifier, t_i is evidence time, b_i is content, and m_i is provenance metadata including the attribution role needed to distinguish target-authored text from quoted, replied-to, or external material. A state $P^{\gamma}_{p,t}$ is not the evidence itself. It is a derived object that answers two distinct questions: (i) what has the person expressed, and (ii) how should those expressions be interpreted at cutoff t?

A state item is represented abstractly as

$$x = (c, \kappa, I, s, U, \omega), \tag{2.2}$$

where c is a semantic claim or behavioral pattern, κ is a coarse type, I is an evidence-supported temporal interval, s is an epistemic/temporal status, $U \subseteq D_p$ is an explicit semantic-support set, and ω is an optional typed temporal-extension witness. The architecture does not standardize one universal ontology for κ; any policy referenced by ω must instead be declared and versioned in the compiler manifest.

The distinction from memory is deliberate. Raw evidence is an archive; retrieval is an access operation; a profile is a compact description; model weights are an execution substrate or a model-bound personalization mechanism. PCS is a versioned interpretation whose temporal and evidential boundaries are first-class.

Object boundary. CMT does not replace evidence storage or retrieval. The archive preserves authored records; a retrieval index provides access; an executor supplies general capability. PCS is the separately versioned *interpretation layer* whose temporal status, support, revisions, and portability are governed as first-class state.

2.2 The three-clock problem

Equation (1.1) exposes three independent version axes, which we call the *three-clock problem*. The term "clock" is conceptual: t is evidence time, while γ and τ may be compiler/model version identifiers rather than wall-clock timestamps. Define

$$\mathcal{V}_p = \{(t, \gamma, \tau) : P^{\gamma}_{p,t} \text{ is executed by } M_\tau\}. \tag{2.3}$$

Most personalization pipelines implicitly collapse at least two clocks: a mutable profile conflates new evidence with reinterpretation, while person-specific weights conflate state with the executor. PCS keeps them separate. Holding (t, γ) fixed while varying τ measures executor improvement over an identical state artifact. Holding (t, τ) fixed while varying γ measures reinterpretation of the same evidence. Holding (γ, τ) fixed while varying t measures evidence-driven state evolution.

Persistence invariant and identity factorization. We impose the following identity invariant:

$$\mathrm{ID}\Big(P^{\gamma}_{p,t}\Big) := \Phi\Big(D^{\le t}_p, \Gamma_\gamma\Big), \tag{2.4}$$

where Φ denotes the committed state-construction and canonicalization procedure, not a claim that semantics are uniquely determined by evidence. Runtime executor version τ is deliberately absent from the arguments of Φ. Equation (2.4) is therefore an *identity factorization*: evidence and compiler changes may induce a new state version; compatible executor substitution does not. The generated output remains a function of the executor and adapter, so identity invariance does not imply behavioral invariance.

The persistent object is therefore a *versioned state lineage*

$$\mathfrak{P}_p = \{P^{\gamma}_{p,t}\}_{(t,\gamma)}, \tag{2.5}$$

not one eternal root hash. Evidence append or compiler replacement may create a new state version; executor replacement at fixed (t, γ) does not. This distinction resolves an otherwise ambiguous use of "persistence": persistence means continuity of an externally governed state lineage, not semantic or byte identity across all updates.

A key consequence is that *artifact identity depends on evidence and compilation, not on the executor*. An executor adapter $\mathcal{A}_\tau$ may change serialization, query-dependent selection, or runtime formatting for compatibility, but it is transient and is not allowed to redefine the persistent person-state artifact.

2.3 Design requirements

A persistent person state should satisfy the following requirements.

1. **Persistence:** an externally stored state lineage survives sessions and executor upgrades.
2. **Temporal fidelity:** old positions remain historically visible without silently remaining current.
3. **Evidence auditability:** every derived item has a finite support path to admissible evidence.

4. **Attribution integrity:** support preserves who authored, quoted, replied to, or supplied each evidence item.
5. **Conservative revision:** unresolved disagreement is preserved rather than forced into one narrative.
6. **No interpretation laundering:** prior derived state cannot silently become canonical evidence for later state.
7. **Executor decoupling:** persistent state identity excludes model-specific executor parameters.
8. **Adapter noninterference:** runtime compatibility logic may project state but may not mint new person-specific evidence or mutate persistent identity.
9. **Version accountability:** emitted state is bound to the compiler manifest, topology, canonicalization rules, and admissible evidence set that produced it.

2.4 State as an intermediate representation

The three-plane decomposition suggests a systems analogy: PCS plays the role of an intermediate representation (IR) between authored evidence and changing execution machinery. The analogy is deliberately limited—human evidence is not source code and a compiler is not an oracle—but it clarifies five useful obligations. First, the IR must be *source-addressable*: claims remain traceable to evidence. Second, it must be *temporally typed*: current, historical, superseded, and unresolved interpretations cannot be collapsed without a witness. Third, it must be *version-explicit*: changing the compiler does not silently overwrite the previous interpretation. Fourth, it must be *backend-independent*: executor-specific serialization belongs in a transient adapter rather than in persistent identity. Finally, it must be *re-materializable*: higher-order state remains rebuildable from evidence-bearing descendants rather than from a chain of prior natural-language roots.

This view exposes two common failure patterns. A lossy recurrent summary can become the only surviving source for its successor, making previous interpretation indistinguishable from evidence. Conversely, a person-specific parameter update can preserve behavior while making the state artifact inseparable from the model architecture that stores it. Section 4 states these limitations formally.

3 Cognitive Merkle Trees

3.1 Evidence commitments and immutable temporal artifacts

CMT partitions historical evidence into ordered, non-overlapping temporal regions and compiles each frozen region into a source-near artifact L_i. New evidence may create later artifacts or occupy a recent frontier; it does not retroactively rewrite the payload of an already committed historical artifact.

For a partition

$$\Pi_t = (B_1, \dots, B_k, F_t), \qquad \bigcup_{i=1}^{k} B_i \cup F_t = D_p^{\le t}, \tag{3.1}$$

with chronologically ordered, non-overlapping frozen blocks B_i and recent frontier F_t, define

$$L_i = C_{\mathrm{leaf},\gamma}(B_i), \qquad i \in \{1, \dots, k\}. \tag{3.2}$$

CMT does not require one universal block-size threshold. The architectural contract is that the partition policy is recorded, frozen blocks preserve provenance, and later evidence cannot silently mutate their committed content.

At the evidence layer, an implementation may commit a canonical serialization by a domain-separated digest such as

$$h(e) = \mathsf{H}(\texttt{CMT-EVIDENCE-v1} \,\|\, \mathrm{canon}(e)). \tag{3.3}$$

For private corpora, the commitment mechanism may be keyed or otherwise access-controlled; the invariant requires stable identity, not public disclosure of source content. Unkeyed hashes of low-entropy or guessable private evidence can leak membership through dictionary attacks, so a public digest is not itself a privacy mechanism.

3.2 Relation-aware conservative reduction

Internal nodes reconcile child items rather than merely concatenate summaries. The public abstraction distinguishes six outcomes:

Same, Extend, Evolve, Contradict, Emerge, and Keep.

Same denotes semantic equivalence; Extend adds compatible detail; Evolve represents a supported time-indexed update; Contradict preserves incompatible claims when evidence does not establish supersession; Emerge permits a higher-order pattern jointly supported by multiple children; and Keep forbids forced collapse when evidence is insufficient. The reducer is therefore conservative by construction: uncertainty expands the retained state rather than licensing unsupported compression.

For children $V_1, \ldots, V_m$, a parent is

$$V = \mathcal{R}_\gamma(V_1, \ldots, V_m; \mathcal{I}), \tag{3.4}$$

where $\mathcal{I}$ denotes the lifecycle invariants. Semantic reduction need not be associative:

$$\mathcal{R}(\mathcal{R}(A, B), C) \neq \mathcal{R}(A, \mathcal{R}(B, C)). \tag{3.5}$$

Consequently, topology is part of the state version. This is not a defect hidden by hashing: it is an explicit property of semantic compilation. A different topology or compiler may emit a different valid state and must receive a different identity.

3.3 No silent temporal persistence

A recurring longitudinal failure is to confuse the last date on which a claim appeared with the date through which it remains valid. A statement can be current at the end of a local artifact yet historical at the lifetime cutoff. Silence is neither confirmation nor reversal.

Let x_i be a state item in artifact L_i whose evidence-supported interval ends at t_i. CMT forbids the inference

$$\mathrm{current}(x_i, t_i) \implies \mathrm{current}(x_i, t), \qquad t > t_i, \tag{3.6}$$

unless the extension carries a typed temporal witness. CMT permits two architectural witness classes: (i) an Evidence witness that names later admissible evidence supporting continued validity, or (ii) a Policy witness that names a persistence rule declared outside the reducer, versioned in the compiler manifest, and scoped to a claim class and, where appropriate, a finite horizon. A reducer may not mint a new policy merely to justify the item it just produced. Stable facts may therefore persist, but persistence is an inspectable assumption rather than an accidental consequence of summarization.

The architectural rule is thus stronger than "remember the last state": *no untyped persistence*. A parent may retain, supersede, or mark a prior item unresolved only after considering later evidence and any predeclared policy witness. The witness itself is committed in lineage. Changing a policy changes the relevant manifest and therefore the state version. This turns temporal extrapolation from an implicit language-model behavior into an auditable systems decision.

3.4 Item-level support and Merkle identity

For a derived item x in node V, let each lineage edge carry a derivation role $\lambda \in \{\textsc{Support}, \textsc{Context}\}$. Let $\mathrm{child}_S(x)$ be the exact child items used as semantic support and $\mathrm{child}_C(x)$ the child items used only as context. Define

$$\mathrm{supp}(x) = \bigcup_{z \in \mathrm{child}_S(x)} \mathrm{supp}(z), \qquad \mathrm{ctx}(x) = \bigcup_{z \in \mathrm{child}_C(x)} \big(\mathrm{supp}(z) \cup \mathrm{ctx}(z)\big), \tag{3.7}$$

with leaf support terminating in evidence identifiers from the corresponding source block. Context is retained in lineage but cannot silently satisfy a support obligation. Parent lineage is computed from the child objects actually used, not inherited from an entire child artifact. This distinction matters whenever an artifact contains quotations, interlocutor text, retrieved background, or irrelevant state items.

Let Γ be a compiler manifest containing, at minimum, the compiler-family identifier, state-schema version, canonicalization version, reduction topology, any *compiler-side* model/provider revision and decoding configuration, and the declared persistence-policy registry. A model used inside C_γ is therefore part of compiler version γ; the replaceable *runtime executor* M_τ is not. The committed payload includes state items' temporal status, support and context edges, attribution roles, relation labels, and typed temporal witnesses; these audit fields are not left outside the commitment. Following the content-commitment role of Merkle-style hashing [16], define the canonical payload digest $d(V) = \mathsf{H}(\mathrm{canon}(\mathrm{payload}(V)))$. A node identity commits to the node class, temporal interval, canonical payload digest, ordered child identities, and manifest digest:

$$h(V) = \mathsf{H}(\texttt{CMT-NODE-v2} \parallel \mathrm{type}(V) \parallel I(V) \parallel d(V) \parallel h(V_1) \cdots h(V_m) \parallel h(\Gamma)). \tag{3.8}$$

Here $\parallel$ denotes an unambiguous typed, length-delimited canonical framing rather than raw string concatenation, and the framing/canonicalization version belongs in Γ. Under the usual collision-resistance assumption, the digest gives a stable commitment to the emitted bytes and ordered derivation inputs; it does not establish semantic entailment. If a compiler uses nondeterministic hosted inference, the emitted payload still receives an unambiguous content identity even when an exact rerun is not guaranteed. Bitwise reproducibility is a stronger property and requires a sufficiently deterministic manifest and execution environment.

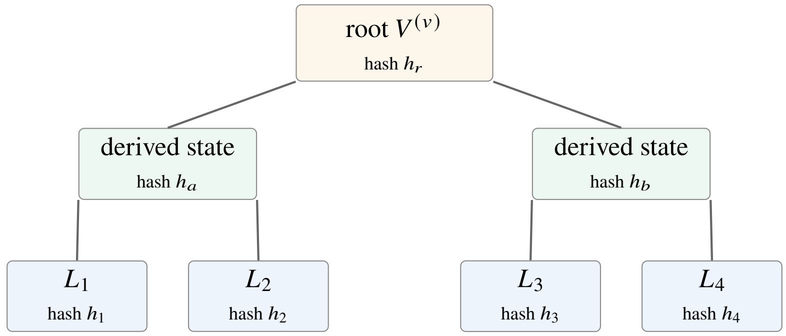

Figure 2: Semantic reduction and cryptographic commitment are separate layers. Each derived item carries a support path to child items and, transitively, authored evidence. Hashes bind artifact identity and ancestry; they do not certify semantic truth.

3.5 Attribution-preserving support

A support identifier is insufficient if the system forgets *whose* statement it identifies or *how* it was used. CMT therefore separates evidence attribution from derivation role. Let $a(e)$ denote an attribution role such as target-authored, quoted-other, replied-to-other, or external-context, and let $\lambda_x(e)$ denote whether e contributes SUPPORT or CONTEXT to item x. A state item attributed to person p must have target-authorized terminal support; third-party material may shape interpretation as context but cannot silently discharge that first-person support requirement. The exported lineage retains both $a(e)$ and $\lambda_x(e)$.

This two-axis provenance is a structural guard against a common failure in social and conversational corpora: a memorable quotation, headline, or interlocutor statement is summarized as if it were the target person's own belief. Attribution integrity does not prove sincerity or correctness of source metadata; it makes both the authorship assumption and the derivational use explicit and auditable.

3.6 Tree semantics, DAG storage, and the recent frontier

The name *Cognitive Merkle Tree* refers to the logical ordered reduction topology. Item-level support can naturally form a DAG when one item supports multiple higher-order items, and content-addressed storage can deduplicate identical artifacts. The architecture therefore does not claim that every physical implementation is a strict pointer tree.

The newest evidence is operationally different from frozen history: it is incomplete by construction and may not justify durable generalization. CMT therefore keeps a recent frontier F_t distinguishable from committed historical artifacts. A root is a versioned materialized view over evidence-bearing descendants plus the admissible frontier; it is not a replacement for the evidence itself.

Critically, the previous root is not used as semantic evidence for the next root. We call this the *no semantic self-feed* rule. If $V^{(v)}$ is the current root and new evidence arrives, a later root $V^{(v+1)}$ is derived from canonical evidence and committed descendants under a new manifest or cutoff, rather than from the natural-language payload of $V^{(v)}$. A previous root may be cached for serving or compared for diagnostics, but it cannot become an unmarked evidence item. This blocks a structural source of copy-of-copy drift and what we call *interpretation laundering*: an earlier model inference acquiring the status of source evidence merely because it survived long enough.

3.7 Corrections, revocation, and deletion

Immutability is a statement about artifact identity, not an instruction to ignore corrections or privacy obligations. A source correction can be represented as a later evidence event that explicitly supersedes an earlier record. Revocation or deletion changes the *admissible evidence set* and produces a new state version; it does not silently rewrite the hash of an old artifact. Every derived item in the reverse support cone of revoked evidence becomes ineligible for reuse and must be removed or rebuilt. Unaffected committed subtrees may retain their identities when their inputs and manifest fields are unchanged. Deployments may additionally make revoked payloads inaccessible or cryptographically erase protected content while retaining only a non-content tombstone or minimum commitment where lawful and necessary for audit. The precise retention policy is deployment-specific and is not a semantic invariant of CMT.

3.8 Reference compilation protocol

CMT is intentionally compiler-agnostic, but a top-level protocol is useful for separating required semantics from implementation freedom. Given admissible evidence $D_p^{\leq t}$, compiler manifest Γ, a partition policy, and an artifact store, a conformant compiler follows the dependency direction *evidence* $\rightarrow$ *local artifacts* $\rightarrow$ *higher-order state*; it never treats the previous root as new source evidence.

Reference protocol: compile$(D_p^{\leq t}, \Gamma)$

1. Canonicalize admissible evidence, assign stable identifiers, and record corrections or revocations as versioned evidence-set changes.
2. Partition evidence into ordered frozen blocks $B_1, \dots, B_k$ and a recent frontier F_t under a recorded partition policy.
3. Reuse a frozen artifact only when its committed input identities and relevant manifest fields are unchanged; otherwise compile a new source-near artifact.
4. Reconcile ordered child items with $\mathcal{R}_\gamma$, enforcing support closure, conservative non-collapse, and the no-untyped-persistence rule.
5. Emit every derived item with exact Support/Context lineage edges, attribution roles, temporal status, and any typed Evidence or predeclared Policy temporal witness.
6. Canonicalize each emitted payload, commit the ordered derivation manifest (including policy registry and compiler-side model revision), and compute its content identity.
7. Emit a root state plus manifest. Runtime executors consume that state only through a compatible executor adapter $\mathcal{A}_\tau$; the adapter is excluded from persistent state identity and is forbidden from minting person-specific evidence.

Figure 3: A minimal reference compilation protocol. The protocol constrains dependency direction and audit contracts without standardizing prompts, ontologies, model routing, or storage engines.

4 Formal Lifecycle Properties

We separate structural guarantees from semantic accuracy. The results below are best read as *lifecycle laws*: contract-preservation properties that a conformant implementation can mechanically test given explicit assumptions. We state them formally to expose the dependency boundary, not to turn natural-language interpretation into a mathematical oracle. The nontrivial architectural claim is that these laws are enforced *together* around one independently versioned person-state object.

Let $\mathcal{E}_{p,t}$ denote the admissible evidence set for person p at cutoff t after any declared correction or revocation policy. Let Γ be the compiler manifest. The structural assumptions are:

(A1) evidence identifiers and canonicalization are stable within a version;

(A2) a leaf Support edge may terminate only in evidence from its source block; context is separately typed;

(A3) a parent item may reference only explicit child items, with every lineage edge typed as Support or Context;

(A4) committed historical artifacts are append-only with respect to later evidence, absent a new correction/revocation version;

(A5) state hashes commit the payload, manifest, and ordered descendants but exclude runtime executor identity;

(A6) temporal extension beyond direct support requires a typed Evidence witness or a Policy witness that refers to a rule predeclared and versioned in Γ; the reducer cannot mint that policy as post hoc justification;

(A7) every evidence reference preserves both attribution role and derivation role;

(A8) derived state is not admissible authored evidence for a later compilation unless explicitly imported as non-authored derived material; and

(A9) a compatible executor adapter may select, order, truncate, or serialize existing state but may neither mutate persistent state identity nor introduce new person-specific claims, evidence, or support identifiers.

Definition 4.1 (Lifecycle configuration)**.** *A runtime configuration is*

$$\Xi_p^{t,\gamma,\tau} = (\mathcal{E}_{p,t}, \Gamma_\gamma, P_{p,t}^\gamma, M_\tau, \mathcal{A}_\tau). \tag{4.1}$$

Lifecycle law	Mechanically testable consequence
Evidence closure	Every semantic support path terminates in the admissible evidence set.
Attribution separation	Third-party context cannot silently satisfy target-person support.
No untyped persistence	Current-state extension requires an evidence or predeclared policy witness.
Temporal non-retroactivity	Appending later evidence does not mutate an already committed historical artifact.
No semantic self-feed	A prior derived root cannot become unmarked authored evidence.
Non-recurrent regenerability	Higher-order state can be rebuilt from evidence-bearing descendants without the prior root payload.
Executor-substitution invariance	Compatible runtime executor replacement leaves persistent state identity unchanged.
Change localization	Unchanged committed subtrees retain identity under append-only growth.
Revocation locality	Only the reverse support cone of revoked evidence is invalidated, modulo manifest changes.

Table 3: The CMT lifecycle laws. They constrain provenance and state evolution; none certifies that a natural-language claim is true.

The principal transitions are append evidence ($t \to t'$), recompile ($\gamma \to \gamma'$), substitute executor ($\tau \to \tau'$), *and* revoke/correct ($\mathcal{E}_{p,t} \to \mathcal{E}'_{p,t}$). *The first, second, and fourth may create a new state artifact; executor substitution alone does not.*

Definition 4.2 (Compatible adapter)**.** *An adapter $\mathcal{A}_\tau$ is compatible with state P if it is semantics-noncreative under A9: every person-specific semantic unit it mounts is a projection of P, and every support or context identifier it exposes already belongs to P's lineage. Query-dependent selection may remove information but may not manufacture new person-specific state. Compatibility constrains state mutation, not output equivalence between executors.*

Definition 4.3 (Persistent cognitive state)**.** *A PCS instance at cutoff t and compiler version γ is a finite, versioned collection $P^{\gamma}_{p,t}$ of state items together with a derivation graph and manifest such that every state item has a temporal interpretation and a finite support path terminating in $\mathcal{E}_{p,t}$. Its identity is defined independently of any executor M_τ.*

Definition 4.4 (Structurally valid state item)**.** *A state item $x = (c, \kappa, I, s, U, \omega)$ is structurally valid at cutoff t if: (i) $U = \operatorname{supp}(x) \subseteq \mathcal{E}_{p,t}$; (ii) its support and context paths are finite and terminate in evidence identifiers with preserved attribution and derivation roles; (iii) any interpretation of x as current beyond the latest directly supporting evidence time carries a valid typed witness under A6; and (iv) no derived-state object is silently relabeled as authored evidence. Structural validity does not imply that the natural-language claim c is semantically correct.*

Proposition 4.1 (Evidence closure)**.** *For every structurally valid root item x at cutoff t, A1–A3 imply*

$$\operatorname{supp}(x) \subseteq \mathcal{E}_{p,t}. \tag{4.2}$$

Proof. Induct on reduction depth. Leaf support is contained in its source block by A2. Each parent support is the union of explicit child supports by Eq. (3.7) and A3. No reduction step can introduce an evidence identifier outside $\mathcal{E}_{p,t}$. □

Proposition 4.2 (Attribution separation)**.** *Under A2, A3, and A7, a root item presented as a claim of person p can be mechanically required to contain target-authorized terminal* Support*; quoted, replied-to, or external material may remain on* Context *edges but cannot silently discharge that first-person support obligation.*

Proof. At leaf level, A7 preserves both attribution and derivation roles. Equation (3.7) propagates only Support edges into $\operatorname{supp}(x)$ while retaining context separately. Because reduction never erases either role, a validator can reject a target-person item lacking target-authorized terminal support even when abundant third-party context is present. □

Proposition 4.3 (No interpretation laundering)**.** *Under A2, A3, and A8, repeated recompilation cannot make the natural-language payload of a prior derived root become authored evidence merely through recurrence. Every admissible authored support path still terminates in $\mathcal{E}_{p,t}$.*

Proof. A8 excludes prior derived state from the authored evidence set. A2 and A3 restrict leaf and parent references to admissible inputs and explicit children. Induction over any number of recompilations therefore preserves termination in admissible evidence rather than in an unmarked prior root payload. □

Proposition 4.4 (Temporal non-retroactivity)**.** *Let L_i be a committed artifact over a frozen source block B_i. Under A1 and A4, appending evidence strictly after the end of B_i does not change $h(L_i)$.*

Proof. The canonical payload, input identities, and manifest fields committed by L_i are fixed. Later evidence may change a higher-order interpretation but is not an input to the already committed artifact. □

Definition 4.5 (Conservative non-collapse)**.** *If available evidence and declared persistence policies do not justify equivalence, compatible extension, or temporal supersession between two items, a valid CMT reduction may preserve both items rather than force a single narrative.*

This is analogous in spirit to belief-revision formalisms that distinguish principled revision from arbitrary overwrite [17]; CMT does not claim that its six relation labels constitute a complete belief-revision logic.

Proposition 4.5 (No-untyped-persistence safety)**.** *Under A6, if a state item x has neither a later-evidence witness nor a predeclared policy witness authorizing extension after time t_x, then a conformant compiler cannot validly promote* $\mathrm{current}(x, t_x)$ *to* $\mathrm{current}(x, t)$ *for any $t > t_x$.*

Proof. Such a promotion requires a typed temporal-extension witness by structural validity. A policy invented by the same reduction step is inadmissible under A6, so absence of both permitted witness classes makes the promoted item structurally invalid. □

Proposition 4.6 (Non-recurrent regenerability)**.** *Suppose a root $V^{(v)}$ is derived from canonical evidence and committed descendants under manifest Γ. Discarding $V^{(v)}$ does not discard the evidence or those descendants, and a new root can be derived without using the natural-language payload of $V^{(v)}$ as semantic input.*

Proof. The root is a materialized interpretation over lower-level evidence-bearing objects rather than their unique carrier. The reference protocol in Fig. 3 defines regeneration from those descendants and the manifest. □

Remark 1 (Regeneration is not byte reproducibility). Non-recurrent regenerability does not imply byte-identical reruns. Bitwise reproducibility additionally requires deterministic compilation, canonical serialization, and a controlled execution environment. Otherwise a rerun is a new emitted artifact with a new content hash.

Proposition 4.7 (Executor-substitution invariance)**.** *If A5 and A9 hold and $P^{\gamma}_{p,t}$ contains no person-specific executor parameter update, replacing a compatible pair $(M_\tau, \mathcal{A}_\tau)$ by $(M_{\tau'}, \mathcal{A}_{\tau'})$ does not change $h(P^{\gamma}_{p,t})$. No person-specific weight retraining is required.*

Proof. Executor and adapter identities are runtime variables in Eq. (1.1); A5 and Eq. (2.4) exclude them from the persistent-state commitment, and A9 forbids the adapter from mutating committed state or minting new support. Replacement can change mounted projections and generated outputs, but not the identity of $P^{\gamma}_{p,t}$. □

Remark 2 (Portability is not behavioral invariance). Executor-neutral state identity does not imply $M_\tau(q \oplus \mathcal{A}_\tau(P)) = M_{\tau'}(q \oplus \mathcal{A}_{\tau'}(P))$. Different compatible executors may reason, calibrate, or verbalize differently. Portability means the person-specific state need not be retrained or redefined merely to substitute the execution substrate.

Separation consequences. Two comparison results—why lossy summary-as-sole-source cannot recover exact evidence lineage, and why model-bound parameter state requires an explicit migration relation across incompatible executors—are useful consequences of the object boundary but are not lifecycle laws of CMT itself. We move their formal statements to Appendix C to keep the main formal section focused on the claimed conformance surface.

Proposition 4.8 (Change localization under append-only growth)**.** *Assume a fixed ordered tree topology over frozen blocks and a new evidence append that changes only the recent frontier or introduces a new rightmost block. Every frozen subtree whose descendant evidence identities and manifest fields are unchanged retains its hash. Only changed frontier artifacts and their ancestor commitments require new identities.*

Proof. Equation (3.8) is content-addressed over payload, ordered child identities, interval, and manifest. A subtree with identical committed inputs therefore has the same identity. Any ancestor whose child identity changes must be recommitted. In a balanced tree the number of structurally affected ancestors is logarithmic in the number of frozen blocks, although semantic recompilation cost at those ancestors remains implementation-dependent. □

Corollary 4.1 (Audit localization)**.** *When two roots share a subtree hash, the committed artifacts beneath that subtree are identical under the hash assumptions. Investigating a version difference can therefore focus on changed subtrees and manifest fields rather than re-auditing all historical artifacts.*

Proposition 4.9 (Revocation-cone locality)**.** *Let $R \subseteq \mathcal{E}_{p,t}$ be revoked evidence. Under explicit item-level support, every derived item whose transitive support intersects R is in the reverse support cone of R and becomes ineligible for reuse. Any committed subtree whose transitive support is disjoint from R and whose manifest fields are unchanged may retain its identity.*

Proof. Items with support intersecting R violate evidence closure with respect to the new admissible set and must be removed or rebuilt. For a disjoint subtree, neither its evidence inputs nor manifest change; Eq. (3.8) therefore yields the same commitment under the hash assumptions. □

Definition 4.6 (Model-evolution dividend)**.** *For fixed state P and evaluation distribution Q, define*

$$\Delta_{\tau\to\tau'}(P) = \mathbb{E}_{q\sim Q}\left[S(M_{\tau'}(q \oplus \mathcal{A}_{\tau'}(P))) - S(M_{\tau}(q \oplus \mathcal{A}_{\tau}(P;q)))\right], \tag{4.3}$$

where S is a task-specific score. A positive value means the same person-state artifact becomes more useful under a stronger compatible executor.

The claims in this section are lifecycle claims. They establish neither semantic entailment nor psychological fidelity. Their purpose is to make a family of failure modes mechanically testable while leaving semantic validity to empirical evaluation.

5 Conformance and Reproducibility Interface

CMT is an architecture, not a single prompt, ontology, model, or production compiler. Reproduction therefore targets observable contracts rather than a private realization. We call an implementation *CMT-conformant for a stated profile* if it exposes enough information to test all of the following:

1. chronological evidence boundaries and a stable evidence identity scheme;
2. immutable committed temporal artifacts plus a distinguishable recent frontier;
3. relation-aware reconciliation that can preserve unresolved alternatives;
4. typed temporal-extension witnesses, with policy witnesses referring only to predeclared, versioned rules;
5. item-level SUPPORT/CONTEXT paths terminating in admissible evidence identifiers with preserved attribution and derivation roles;
6. content-addressed artifact identities that bind payload, ordered descendants, and a compiler manifest including relevant compiler-side model revision and policy registry;
7. regeneration of higher-order state without using the previous root as semantic evidence;
8. revocation-cone invalidation under a changed admissible evidence set; and
9. an executor-independent persistent state identity plus adapter noninterference.

We intentionally do not standardize a universal state-field ontology, compiler instruction program, partition threshold, retrieval policy, runtime adapter, or serving layout. These choices can vary while preserving the lifecycle contract. This is analogous to a systems specification that standardizes externally testable semantics without requiring one scheduler or storage engine.

The arXiv source package accompanying this paper includes a small deterministic structural checker plus positive and adversarial JSON fixtures. The checker validates canonical artifact hashes, support closure, support-versus-context roles, target-attribution requirements, typed temporal-witness references, graph cycles, and forbidden self-feed references. It intentionally does not judge whether a natural-language claim is semantically entailed; that boundary is itself part of the specification.

A useful reporting distinction follows. *Conformance reproducibility* asks whether another implementation can satisfy and mechanically test the same lifecycle properties. *Artifact reproducibility* asks whether the identical bytes can be regenerated; this requires a deterministic compiler realization and is stricter. *Behavioral reproducibility* asks whether controlled evaluations yield comparable effects. Conflating the three obscures what an architecture paper can and cannot establish.

6 CMT-L5: Longitudinal Evaluation Corpus

To study persistent person state without contaminating the future, we construct CMT-L5 from five public longitudinal author streams. Retweets are excluded. Replies are retained when authored by the target person. For quote posts, the target author's own commentary is retained while quoted third-party text is excluded from author evidence. Rows are stably ordered by timestamp and source identifier. No generative transformation is applied during corpus export.

Because multilingual social text has no model-independent tokenizer, corpus size is reported in *packing units* produced by the deterministic sizing rule used during corpus construction: CJK characters contribute one unit, while non-CJK text contributes approximately one unit per four characters. These values are packing estimates, not claims about any model tokenizer.

Person	Posts	Packing units	Span (days)	Quote posts	Thread metadata
P1	13,963	1,900,233	1,758	857	partial
P2	5,262	320,263	1,697	188	available
P3	6,152	645,160	390	477	partial
P4	2,245	317,822	1,380	172	available
P5	1,699	332,082	631	589	partial
Total	29,321	3,515,560	–	2,283	–

Table 4: CMT-L5 corpus statistics. Authors are anonymized in the paper; observation spans range from approximately 1.1 to 4.8 years.

6.1 Deterministic temporal splits and leakage contract

For each person, 70/80/90% cutoffs are defined by cumulative packing mass. For cutoff t,

$$\text{TRAIN}(t) = \{e : t(e) \leq t\}, \qquad \text{FUTURE}(t) = \{e : t(e) > t\}. \tag{6.1}$$

No future evidence may contribute to person state, retrieval, examples, query planning, state summaries, compiler demonstrations, or representation selection. Candidate representations are frozen before future evidence is opened. Future evidence may then be used to construct and score future-facing tasks, but never to revise a candidate state for that cutoff.

Cutoff	Train posts	Train units	Future posts	Future units
70%	23,244	2,461,481	6,077	1,054,079
80%	25,696	2,814,012	3,625	701,548
90%	27,773	3,169,771	1,548	345,789

Table 5: Aggregate temporal splits across five people. The 90% split is the strictest future-held-out condition.

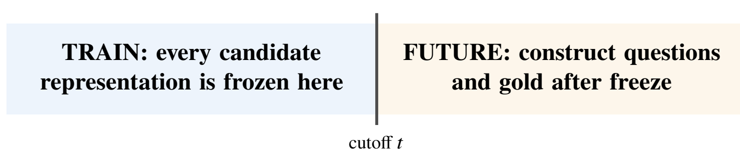

Figure 4: Leakage contract. Future evidence is unavailable to candidate representations and is opened only after state construction, first for locked task construction and then for scoring. Candidate outputs may not influence case retention.

7 Invariant Stress Test: Unsupported Temporal-Horizon Extension

Before asking whether CMT improves future behavior, we isolate one narrower contract violation: *unsupported temporal-horizon extension*. A compiler may observe a dated assertion, compress it into a durable category, and later treat it as current even though neither later evidence nor an explicit persistence rule justifies extension. Repeated compaction makes this failure especially plausible because a dated statement can become an undated summary.

We compare two state interpretations over the same one-author evidence snapshot containing 5,250 source items. The first is *recall-first*: it favors retention and compact durable representation. The second is *temporally constrained*: dated evidence remains bounded unless later evidence or an explicit persistence rule supports extension, and unresolved status is preserved rather than projected forward.

This is an invariant stress test, not a measure of person-simulation accuracy. The metric asks whether a state item violates an explicit temporal contract; it does not ask whether the entire representation is psychologically or behaviorally correct.

The primary diagnostic is deliberately narrow. The heuristic proxy flags durable stable-knowledge and procedural items whose interpretation extends materially beyond dated support without later evidence or a recorded persistence rule establishing continuity. In the recall-first state, 60 stable-knowledge items and 10 procedural items are flagged. In the temporally constrained state, the corresponding counts are 0 and 1.

State interpretation	Stable knowledge	Procedural	Total flagged
Recall-first	60	10	70
Temporally constrained	0	1	1

Table 6: Unsupported-horizon diagnostic on the single stress-test author. The $70 \to 1$ change is a 98.6% reduction in *heuristic contract flags*, not a behavioral-accuracy estimate.

Within the two inspected categories, the flagged fraction changes from $70/108 = 64.8\%$ to $1/94 = 1.1\%$. The denominators differ because the constrained compiler also changes state cardinality, so neither the percentage nor the 70-to-1 count is a controlled accuracy estimate. The remaining procedural flag is consistent with a potentially durable decision rule and is not automatically an error. Full category-cardinality diagnostics are retained in Appendix B only as an audit trail, not as a quality metric.

Because the proxy is heuristic, the snapshot contains one author, and the comparison exercises one compiler realization, the result should be read only as evidence that silent temporal extension is a concrete, observable failure mode that can be isolated by contract. It does not establish behavioral superiority, compiler-family robustness, or population-level effect and should never be summarized as "98.6% more accurate."

8 Future-Held-Out Behavioral Evaluation

The stronger behavioral hypothesis is falsifiable: holding evidence, executor, and representation budget fixed, does a CMT state better predict a person's later authored judgment than alternative representations constructed from the same pre-cutoff evidence?

8.1 Pre-specified hypotheses

Before any completed behavioral comparison is interpreted, the evaluation plan should lock and separate claims that are often collapsed into one headline number.

- **H1 – Temporal fidelity:** CMT produces fewer unsupported current-state extensions than baselines under blinded adjudication.
- **H2 – Future judgment:** at matched mounted-state budget and fixed executor, CMT improves paired future-held-out judgment score J.
- **H3 – Auditability:** a randomly sampled answer-relevant state claim can be traced to admissible evidence with higher exact lineage completeness and lower audit effort.
- **H4 – Model-evolution dividend:** holding P fixed, a stronger compatible executor improves task score without rebuilding person state.
- **H5 – Efficiency:** any quality gain is reported jointly with construction cost, mounted tokens, update latency, and query latency rather than treated as free.

H1 and H3 are primarily state-contract questions; H2 and H4 are behavioral questions. Failure on one does not logically imply failure on the others.

8.2 Single-backbone isolation and budget matching

Let M be one frozen executor and $R_j(D_p^{\le t})$ representation method j. We compare

$$\widehat{y}_{p,q,j} = M\Big(q \oplus \mathcal{A}(R_j(D_p^{\le t}); q)\Big) . \tag{8.1}$$

The executor is held constant across conditions so that representation, rather than foundation-model capability, is the principal manipulated variable. All methods receive the same pre-cutoff evidence universe. Candidate prompts hide method names.

Representation comparisons must also control capacity. We therefore require a declared mounted-state budget B and report both the maximum budget and realized mounted size for every condition. If a representation requires query-time selection to fit B, that selection mechanism is part of the representation method and its cost must be reported. Compiler-side model calls, latency, token cost, and maximum generation ceilings are secondary outcomes rather than silently free resources. A headline quality difference cannot be attributed purely to semantic operators when the winning representation is materially larger or receives materially greater compiler compute; a capacity-equalized sensitivity analysis is required in that case.

8.3 Representation families

The protocol includes seven conditions: (1) no person-specific state, (2) recent raw context, (3) lexical retrieval, (4) independent temporal artifacts without lifetime reconciliation, (5) global flat consolidation, (6) recurrent summary mutation, and (7) CMT.

Method	Persistent	Temporal history	Item lineage	Executor-neutral state	Prior-root reuse	Main limitation
No state	–	–	–	yes	no	no person evidence
Recent raw	–	implicit	raw	yes	no	bounded context
Lexical retrieval	index	implicit	raw	yes	no	query-local, no coherent lifetime state
Independent artifacts	yes	yes	local	yes	no	no lifetime reconciliation
Flat consolidation	yes	weak	coarse	yes	no	may collapse temporal distinctions
Recurrent summary	yes	partial	weak	yes	yes	accumulated rewrite drift
CMT	yes	explicit	transitive	yes	no	semantic compilation remains fallible

Table 7: Representation families in the future-held-out evaluation. The table describes intended architectural properties, not measured performance.

8.4 Task construction and scoring

At a locked cutoff, candidate systems receive only pre-cutoff evidence. Future posts are opened only after candidate representations are frozen. Future evidence is converted into neutral questions about judgments, preferences, decisions, predictions, causal explanations, or calibrated assessments. Questions that merely ask for factual recall, contain wording that reveals the later stance, or require private information not present in the authored record are rejected before candidate answers are produced.

For each case, the gold record captures the later stance, explicit action where available, reasoning points, key variables, and calibration supported by selected future authored evidence. Each candidate is scored on five 0–4 dimensions:

$$J = \frac{1}{20}\left(s_{\text{stance}} + s_{\text{action}} + s_{\text{reasoning}} + s_{\text{variables}} + s_{\text{calibration}}\right). \tag{8.2}$$

Unsupported assertions and provenance failures are reported separately rather than hidden inside a tunable penalty. System identities are randomized during judging.

The preferred primary judge is independent of the candidate executor: human raters, a cross-family evaluator, or both. If two blind passes use the same judge model, they measure randomized-pass repeatability rather than inter-model agreement and must be labeled accordingly. Inter-rater agreement, pass-specific means, and judge sensitivity should be reported where feasible.

8.5 Mechanism ablations

A positive end-to-end result is insufficient to identify which lifecycle mechanism matters. We therefore specify five ablations that preserve the same executor and evidence universe: (i) remove the no-silent-persistence constraint; (ii) replace item-level lineage with artifact-level provenance; (iii) feed the previous root into the next root, introducing recurrent mutation; (iv) flatten the lifetime state while preserving the same mounted budget; and (v) merge the recent frontier into frozen history immediately. These ablations map directly onto CMT's claimed mechanisms and should be reported even when some reduce performance.

8.6 Contamination and case-construction controls

Public-author evaluation is vulnerable to pretrained memorization and benchmark-selection effects. The protocol therefore includes a no-state control, anonymized person identifiers, exact-phrase overlap checks between questions and future gold evidence, and case accounting before answer generation. Candidate outputs must not influence which future cases are retained. For a stronger release, a subset of questions should be independently reconstructed by a second annotator from the locked future evidence, with disagreements adjudicated before system identities are revealed.

8.7 Cluster-aware inference

Questions from the same person are correlated and cannot be treated as independent human subjects. For CMT versus baseline B, define the paired difference

$$d_{p,q} = J_{\mathrm{CMT},p,q} - J_{B,p,q}. \tag{8.3}$$

Headline uncertainty should resample at the person level first and questions within person second, while preserving the pair on the same question. With only five person clusters, an exact $2^5 = 32$ person-block sign-flip sensitivity analysis is intentionally coarse; fine-grained population-level significance claims would be inappropriate. Per-person effects and the full paired distribution are more informative than a single pooled p-value.

8.8 Reporting rule

This report does not claim a completed behavioral-superiority result. The locked statistical analysis plan and execution artifacts are versioned separately from this architectural claim; incomplete or failed transport runs are not evidence. A future result enters the evidence base only if the locked evaluation matrix passes deterministic checks for case count, system count, score ranges, train/future separation, budget compliance, provenance closure, and complete accounting of excluded cases. The primary analysis, ablations, and judge protocol should be fixed before system labels are unblinded. Incomplete transport runs, partial matrices, selective case deletion, or result-dependent reruns are excluded. A clean negative result is more informative than a contaminated positive one.

9 Relationship to Prior Work

9.1 Long-term memory as storage, management, and operating substrate

MemGPT frames external context management as an operating-system-like problem; MemoryBank and Mem0 address persistent storage and retrieval; A-MEM dynamically organizes memory as linked notes; MemoryOS and MemOS elevate memory management to a dedicated systems layer; and MemForest emphasizes write-efficient temporal indexing [5, 6, 7, 18, 19, 20, 8]. These systems establish that memory deserves explicit architecture rather than ad hoc prompt stuffing. CMT changes the object boundary one step further: it treats *person-state interpretation* as distinct from both the evidence store and the execution model.

9.2 Temporal and structured memory

Zep uses a temporally aware knowledge graph to maintain historical relationships, while RAPTOR and hierarchical memory systems show how structured abstraction can improve access to large histories [21, 22, 8]. DCPM introduces diachronic belief trajectories and higher-order schemas, and Kumiho grounds versioned memory in formal belief-revision semantics [11, 14]. CMT is closest in spirit to this family but makes a different contract primary: historical artifacts are immutable, temporal extension requires a typed evidence or predeclared-policy witness, and the lifetime state is independently versioned from both compiler and executor.

9.3 Person simulation and structured person understanding

Generative Agents show behaviorally plausible simulations grounded in memory streams [23]; self-report-grounded agents show that language models can simulate individuals across tasks [24]. LaMP and dynamic profiling benchmarks study personalized response generation, while PersonaTree organizes person understanding into a structured lifecycle memory with explicit support paths [25, 9, 10]. These works make the lifecycle question sharper: once a system derives durable person-level claims, what exactly persists when evidence, the compiler, or the executor changes? PCS is proposed as that persistent systems object.

9.4 Benchmarks for long-horizon memory

LongMemEval and RealMem expose failures that only appear across sessions and evolving projects [26, 27]. CMT-L5 differs from a general memory benchmark in that its split is designed around *future-authored person judgments*: candidate representations are frozen at a chronological cutoff, and later authored evidence is reserved for construction and scoring. The design is intended to evaluate person-state representation rather than raw conversational recall.

9.5 Provenance, versioning, and portable memory

W3C PROV provides a general vocabulary for derivation and versioning [28], while database provenance formalizes why- and where-style lineage as a first-class systems concern [29]. Portable Agent Memory uses a Merkle-DAG for transferable memory state, while MemLineage couples cryptographic provenance to derivation lineage for memory security [12, 13]. Event-sourced agent architectures similarly separate append-only history from materialized views and verification [30]; classical materialized-view maintenance makes the same base-data/view distinction in databases [15]. CMT does not claim novelty for hashes, append-only logs, provenance, or materialized views in isolation. Its narrower contribution is to bind these systems ideas to a specific semantic lifecycle for longitudinal authored-person state.

9.6 Long context and model/harness separation

Long-context capacity does not by itself guarantee reliable use of distant information [1, 2]. Recent harness work increasingly distinguishes model capability from the systems substrate that executes and constrains it [31]. Kimi K3 illustrates how quickly the executor frontier can move in context length and agentic capability [3]. CMT applies the same separation pressure to personalization: the person-specific artifact should remain externally inspectable while executors improve or are replaced.

10 Novelty and Falsification Boundaries

A reviewer can reasonably ask whether CMT is merely hierarchical summarization plus hashes. If that were the claim, the contribution would be incremental. The paper instead proposes a *state identity contract*: what persists in longitudinal personalization is an independently versioned person-state IR, while evidence, compilation, and execution remain separately addressable. Four statements should therefore not be conflated.

1. **The object-boundary thesis:** evidence, person-state interpretation, and execution are different systems objects. PCS names the missing middle object.
2. **The persistence invariant:** state identity factors through admissible evidence and compiler version, not runtime executor identity. This is the architectural consequence of the three-clock separation in Eq. (2.4).
3. **The CMT conformance claim:** CMT is one concrete realization of that principle using immutable temporal artifacts, typed temporal witnesses, two-axis provenance, non-recurrent regeneration, revocation locality, and content-addressed derivation identity. These are mechanically testable structural claims.
4. **The behavioral hypothesis:** under controlled budgets and a fixed executor, CMT will predict future person judgments better than alternative representations. This is empirical and can be false. The present report does not claim that it has been established.

The novelty is therefore neither Merkle hashing nor hierarchical memory in isolation. Merkle commitments are accountability machinery, not a semantic method. The closest contemporary systems already contain important pieces of the design space: PersonaTree has person-level lifecycle structure and support paths; Kumiho has versioned belief revision; Zep has temporal graph memory; Portable Agent Memory has cryptographically verifiable transfer; MemLineage has derivation-aware provenance; and MemForest has temporal hierarchical maintenance [10, 14, 21, 12, 13, 8]. CMT's narrower contribution is to make *executor-independent state identity plus its temporal/provenance lifecycle* the primary abstraction boundary. A flat, graph-native, or future architecture that satisfies the same identity factorization and lifecycle laws could implement the broader PCS abstraction. The tree is a reference reduction topology, not the definition of PCS.

This framing makes falsification concrete. The behavioral hypothesis is weakened if budget-matched flat or retrieval baselines consistently match or outperform CMT while satisfying the same lifecycle laws at lower cost. A claimed CMT implementation is structurally nonconformant if semantic support escapes the admissible evidence set, context is laundered into target support, temporal extension lacks a valid witness, committed historical artifacts mutate without a new version, prior-root text becomes untracked pseudo-evidence, or compatible executor replacement changes persistent state identity. These are intentionally stronger failure conditions than "the hash verifies."

11 Threats to Validity, Safety, and Ethics

Documented cognition is incomplete. Authored text is a selective behavioral trace. Silence is not evidence of absence; private beliefs are not observable; and a person may write strategically, humorously, inconsistently, collaboratively, or for a specific audience. Some text attributed to an account may also be assisted or generated by others. CMT can represent only the provenance-bearing evidence it is given.

Semantic compilation remains fallible. Evidence closure prevents invented support identifiers from entering lineage but does not guarantee semantic entailment. Relation labels, abstractions, and temporal statuses may still be wrong. Provenance is a prerequisite for review, not a substitute for semantic review.

Persistence policies can become a loophole. A system could satisfy the letter of temporal witnessing while declaring policies so broad that almost every old claim remains current. CMT therefore requires policy witnesses to be predeclared, versioned, scoped, and auditable rather than minted by the reducer. Empirical reports should disclose how often current-state extensions rely on later evidence versus policy, and should treat unusually broad or unbounded policies as a separate sensitivity analysis.

Compiler power can confound representation comparisons. A CMT compiler that receives stronger models, more tokens, or substantially more offline compute than baselines can win for reasons unrelated to representation. Controlled evaluations should match or disclose compiler resources and separately report construction cost.

The invariant stress test is not independent ground truth. The 98.6% reduction in Section 7 uses a heuristic horizon-extension proxy on one author. It demonstrates a conformance failure mode but cannot establish general person-modeling quality.

Five people are not a population. CMT-L5 is longitudinally deep but demographically and topically narrow, with substantial finance/technology discourse. Broad claims require more people, languages, occupations, source types, private-consent cohorts, and non-market decision domains. Person-clustered inference is essential even within the present corpus.

Public-corpus contamination. Public authors may have appeared in foundation-model pretraining. A no-person-state control can reveal some memorization, but content fingerprints may remain identifiable even after names and source IDs are hidden. This limits claims about pure inference from the provided state.

Task-construction bias. Future evidence is necessarily visible to the process that constructs future-facing evaluation questions. The candidate states remain sealed, but benchmark designers can still select convenient cases. Locked construction rules, blinded answer generation, complete case accounting, and release of case-level provenance are necessary to reduce this risk.

Implementation plurality and adapter power. CMT defines architecture-level invariants rather than one canonical compiler. Different valid implementations can produce different semantic states. A runtime adapter can also become an undeclared second compiler if allowed to inject person-specific interpretation. Comparative work should therefore report compiler manifest, compiler-side model revision, policy registry, topology, artifact identities, adapter contract, realized mounted size, budgets, and evaluation conditions; adapters that create new person-specific claims are nonconformant rather than merely "stronger."

Conservative state can grow. Preserving contradiction, uncertainty, and historical alternatives can make state larger than an aggressively collapsing summary. CMT does not claim sublinear semantic-state size or free maintenance. Practical systems may need budget-aware projection, archival tiers, or additional compaction, but those optimizations must preserve support and temporal contracts. Construction cost, update cost, root size, and mounted size are therefore part of the evaluation rather than hidden implementation details.

Privacy, consent, revocation, and impersonation. Persistent person modeling can create risks of unauthorized profiling, reputational harm, manipulation, and posthumous representation. Appropriate deployments should require lawful provenance, consent or authorization where applicable, access controls, revocation/deletion pathways, and clear signaling that generated behavior is a simulation rather than the living person. A public hash should not be treated as a privacy mechanism.

Identity and consciousness. Behavioral continuity does not establish subjective continuity, consciousness transfer, or philosophical identity. CMT is a data-and-execution architecture. Whether a digital process is "the same person" lies outside the technical claims of this paper.

12 Discussion: Persistent State, Replaceable Intelligence

The most consequential implication of CMT is not any particular tree shape. It is the inversion of what a personalized AI system treats as durable. Conventional systems often make the model the long-lived object and user information disposable context. The persistence invariant reverses that default: the person-specific state lineage is durable, while models are replaceable processors.

The three-axis version space sharpens this idea. Evidence can grow from t_1 to t_2 without changing the executor. A better compiler can produce $P^{\gamma_2}_{p,t}$ from the same evidence while preserving the older state version $P^{\gamma_1}_{p,t}$. A new executor M_{τ_2} can run the same state identity without person-specific retraining. These interventions separate three questions that are usually entangled: did the person evidence change, did our interpretation of that evidence change, or did the machine executing the interpretation improve?

This suggests viewing persistent cognitive state as a model-independent intermediate representation for person-specific computation. The analogy is limited—human evidence is neither source code nor a complete specification—but the systems property is useful: the durable artifact sits between raw source evidence and changing execution machinery. Its value comes precisely from being inspectable, challengeable, replaceable, and re-executable.

The architecture also reframes memory. Raw evidence remains canonical memory. Retrieval remains access. The root is a compiled materialized view. Treating a root as the only truth would be a category error; it is useful because it can be discarded and regenerated from evidence-bearing descendants, with its assumptions and support still visible. In that sense, CMT is closer to a versioned semantic build system than to a database of remembered sentences: evidence is source, the compiler emits an inspectable IR, and executors consume the IR through compatibility adapters.

For long-horizon human–machine systems, this separation is a prerequisite rather than a proof of continuity. A person's authored evidence is not itself a person, and a compiled state is not consciousness. Yet without a persistent, auditable person-specific object outside model weights, every model transition restarts the representation problem. With such an object, advances in machine intelligence can accumulate around a stable, versioned human-specific state rather than replacing it.

Relation to the broader BootLoader Labs program. The program-level vocabulary can therefore be mapped cleanly onto the systems decomposition in Eq. (1.1): *Human Image* corresponds to the portable person-state object formalized here as PCS; *Human Emulator* corresponds to executing that state through a compatible adapter on replaceable intelligence; and *Human Continuity* denotes the longer-horizon synchronization process that would repeatedly advance evidence and state versions over time. This paper formalizes the persistent-state layer and its execution boundary. It does not claim consciousness transfer, identity preservation, or biological–digital continuity.

More importantly, the separation creates a research program with orthogonal axes rather than one monolithic personalization score. Compiler research can improve semantic fidelity at fixed evidence and executor. Representation research can test temporal, provenance, and compression choices at fixed executor. Model research can measure the model-evolution dividend on a frozen state. Governance research can study correction, revocation, and selective disclosure without changing the meaning of model capability. The value of PCS is therefore not that it predicts one benchmark outcome in advance, but that it makes these interventions independently measurable.

13 Conclusion

We introduced persistent cognitive state as an independently versioned systems object and Cognitive Merkle Trees as a reference architecture for realizing it. The core separation is simple: *evidence is memory; state is interpretation; intelligence is execution*. The corresponding persistence invariant is an identity constraint: a person-state version is determined by admissible evidence and compilation, while compatible runtime executors remain replaceable. CMT makes that boundary operational through immutable temporal artifacts, conservative reconciliation, typed temporal witnesses, support-versus-context lineage, content-addressed commitments, non-recurrent regeneration, revocation locality, and executor-substitution invariance.

The empirical evidence is deliberately bounded. CMT-L5 contains 29,321 authored posts across five longitudinal individuals with strict future-held-out splits. On one stress-test author, a heuristic unsupported-horizon diagnostic falls from 70 flags to 1 under the temporal contract; this isolates a concrete lifecycle failure mode without establishing behavioral superiority. The stronger behavioral claim remains reserved for the locked, budget-matched, fixed-executor future-held-out evaluation.

The broader thesis is that person-specific state should not have to die with a model version. Evidence can grow. Compilers can improve. Executors can be replaced. State versions can be superseded without erasing their lineage. If longitudinal personalization is to survive rapid model turnover, the durable research object should be the versioned, auditable state lineage across all three clocks. In BootLoader Labs' program vocabulary, this formal state object is the technical core of a Human Image, while Human Emulator and Human Continuity describe progressively broader execution and synchronization layers rather than additional claims proved here.

A Exact CMT-L5 Split Manifest

Person	Split	Train posts	Train units	Future posts	Future units
P1	70	11,889	1,330,278	2,074	569,955
P1	80	12,762	1,520,830	1,201	379,403
P1	90	13,529	1,715,567	434	184,666
P2	70	4,345	224,283	917	95,980
P2	80	4,681	256,329	581	63,934
P2	90	4,974	288,238	288	32,025
P3	70	4,095	451,648	2,057	193,512
P3	80	4,923	516,143	1,229	129,017
P3	90	5,638	580,653	514	64,507
P4	70	1,582	222,794	663	95,028
P4	80	1,847	254,922	398	62,900
P4	90	2,049	286,117	196	31,705
P5	70	1,333	232,478	366	99,604
P5	80	1,483	265,788	216	66,294
P5	90	1,583	299,196	116	32,886

B Invariant Stress-Test Details

The stress test compares two derived states built from the same 5,250-item evidence snapshot. The horizon-extension figure is a heuristic conformance proxy, not a human label of cognitive truth. A flagged item is one whose durable interpretation materially extends beyond dated support without later evidence or an explicit persistence rule establishing continuity. Reductions in state cardinality are not inherently desirable; the inventory diagnostics below are retained only to make the structural effect inspectable.

Derived-state diagnostic	Recall-first	Temporal-safe	Change
Stable knowledge items	88	75	−14.8%
Decision procedures	20	19	−5.0%
Mental models	19	13	−31.6%
Stance timeline items	58	37	−36.2%
Dated facts	67	27	−59.7%
Quoted / not-author candidates	66	17	−74.2%
Epistemic boundaries	23	19	−17.4%

These are compiler-output diagnostics rather than corpus row counts or accuracy labels. In particular, the "quoted / not-author" category denotes candidate derived statements rejected or retained under attribution handling; it should not be read as a count of third-party text admitted into CMT-L5 author evidence.

A representative failure pattern contains both an older position and later evidence that the author exited or revised the position. The recall-first interpretation retains the older state as effectively current because it is durable and salient. The temporal-safe interpretation preserves the older state as historical and treats the later evidence as a distinct evolution. The failure is therefore not evidence recall—both periods are present—but state interpretation.

C Separation Results for Recurrent and Model-Bound State

The following results support the object-boundary argument but are separated from the main lifecycle-law sequence because they characterize alternative state carriers rather than CMT conformance itself.

Proposition C.1 (Lossy recurrent summaries cannot be self-auditing without retained source)**.** *Let $S_{k+1} = f(S_k, E_{k+1})$ be a recurrent summary process in which S_k is the only retained representation of earlier evidence and f is non-injective with respect to that earlier evidence. Then exact evidence-level reconstruction of the support of arbitrary claims in S_{k+1} is impossible from S_{k+1} alone.*

Proof. Because f is non-injective, distinct earlier evidence histories can map to the same retained summary state. A later observer given only that summary cannot identify which history generated it, so exact evidence-level lineage for arbitrary claims is not recoverable. Retaining an external archive or explicit lineage avoids the impossibility; the result targets summary-as-sole-source designs, not recurrent computation per se. □

Proposition C.2 (Model-bound state requires migration for executor substitution)**.** *Suppose the only person-specific state is a parameter delta Δ_τ defined in the parameter space of executor M_τ. For an executor $M_{\tau'}$ with an incompatible parameterization, direct substitution cannot preserve executable state without an explicit migration map $g_{\tau\to\tau'}(\Delta_\tau)$. The migrated object must therefore have an explicit version/identity relation to the source representation.*

Proof. The coordinates and operational meaning of Δ_τ are defined relative to M_τ. Without a map into the parameter space of $M_{\tau'}$, the delta is not directly executable there. A migration introduces an additional transformation whose semantics and identity relation must be versioned. □

D CMT Conformance Checklist

A research implementation is sufficient to evaluate the architecture-level claims in this paper if it provides an externally testable interface for:

- deterministic chronological evidence ordering and future-held-out splits;
- stable evidence identifiers and explicit admissible evidence sets;
- immutable committed temporal artifacts with explicit evidence intervals;
- a distinguishable recent frontier;
- semantic reconciliation capable of preserving equivalence, compatible extension, evolution, contradiction, higher-order emergence, and unresolved uniqueness;
- typed temporal-extension witnesses, with policy witnesses restricted to predeclared versioned rules;
- exact item-level Support/Context paths terminating in admissible evidence identifiers with attribution and derivation roles;
- content-addressed artifact identities binding payload, ordered descendants, compiler manifest, and policy registry;
- higher-order regeneration without using the previous root as semantic evidence;
- revocation-cone invalidation and reuse of unaffected descendants where permitted;
- executor-independent persistent state identity plus adapter noninterference; and
- fixed-executor, budget-matched evaluation when making behavioral representation claims.

No private production prompt, proprietary field schema, partition threshold, model-routing policy, or serving layout is required to satisfy this research interface, because none is itself a claimed invariant of CMT.

E Minimal Interchange Schema

A CMT implementation may use any internal ontology, but an exported state item should expose enough information to reconstruct the lifecycle contract. Table 8 gives a minimal interchange surface.

Field	Required semantics
item identifier	stable identity within the emitted artifact
claim / pattern	human-inspectable semantic payload
type	coarse declared category; ontology is implementation-specific
evidence interval	time span directly supported by evidence
status	e.g., current, historical, superseded, unresolved, or implementation-defined equivalent
lineage edges	exact evidence or child-item IDs, each typed as SUPPORT or CONTEXT
attribution role	target-authored, quoted-other, replied-to-other, external-context, or implementation-defined equivalent
relation	reduction relation such as Same, Extend, Evolve, Contradict, Emerge, or Keep
temporal witness	EVIDENCE references or a predeclared POLICY identifier when validity extends beyond direct support
manifest digest	compiler/schema/canonicalization/topology/policy-registry identity needed to interpret the artifact
artifact hash	content-addressed commitment to payload and derivation identity
state-version parent	optional prior state-lineage identifier; metadata only, never semantic evidence

Table 8: Minimal interchange semantics. Field names and serialization format are not standardized; the audit meaning is.

F Reference Structural Checker

The source package contains `cmt_conformance.py`, a deterministic checker for the minimal interchange surface, together with positive and adversarial JSON fixtures. Given exported evidence and state JSON, it verifies canonical content hashes, support closure, SUPPORT/CONTEXT edge roles, target-attribution requirements, typed temporal-witness references, duplicate evidence identifiers, graph cycles, and explicit rejection of derived-root self-feed. Passing the checker establishes only structural conformance to the checked contracts. It does not establish natural-language entailment, psychological fidelity, cryptographic key management, or behavioral superiority.

G Behavioral Evaluation Lock Checklist

Before a behavioral result is treated as evidence for H2 or H4, the following items should be frozen and machine-checkable: chronological cutoffs; admissible evidence hashes; representation definitions; mounted-state budget; executor version and decoding settings; compiler resource accounting; complete case manifest; question-generation rules; exclusion rules; judge identities or human-rater protocol; primary metric; ablation matrix; and person-clustered inference procedure. After answer generation begins, any deviation should be logged as a new evaluation version rather than silently replacing the locked run.

H Data Statement

The corpus is constructed from public authored social-media material and is anonymized in the paper. Public availability does not eliminate privacy or contextual-integrity concerns. A verbatim release may be constrained by platform terms, deletion requests, author privacy, or later source removal. Where redistribution is not appropriate, a reproducibility package can instead contain deterministic construction code, source identifiers where terms permit, hashes or commitments, aggregate statistics, split manifests, and evaluation code without republishing full source text. The benchmark should not be used to impersonate individuals or infer sensitive private attributes.

References

[1] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. "Lost in the Middle: How Language Models Use Long Contexts". In: *arXiv preprint arXiv:2307.03172* (2023).

[2] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. "RULER: What's the Real Context Size of Your Long-Context Language Models?" In: *arXiv preprint arXiv:2404.06654* (2024).

[3] Kimi Team. "Kimi K3: Open Frontier Intelligence". In: *arXiv preprint arXiv:2607.24653* (2026).

[4] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.

[5] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. "MemGPT: Towards LLMs as Operating Systems". In: *arXiv preprint arXiv:2310.08560* (2023).

[6] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. "MemoryBank: Enhancing Large Language Models with Long-Term Memory". In: *arXiv preprint arXiv:2305.10250* (2023).

[7] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory". In: *arXiv preprint arXiv:2504.19413* (2025).

[8] Han Chen et al. "MemForest: An Efficient Agent Memory System with Hierarchical Temporal Indexing". In: *arXiv preprint arXiv:2605.23986* (2026).

[9] Bowen Jiang et al. "Know Me, Respond to Me: Benchmarking LLMs for Dynamic User Profiling and Personalized Responses at Scale". In: *Conference on Language Modeling (COLM)*. 2025.

[10] Yubo Hou, Jingwei Song, Hongbo Zhang, Zhisheng Chen, Bang Xiao, Tao Wan, and Zengchang Qin. "PersonaTree: Structured Lifecycle Memory for Person Understanding in LLM Agents". In: *arXiv preprint arXiv:2606.04780* (2026).

[11] Tianxiang Fei, Mingyang Song, Mao Zheng, and Xiang Yu. "Memory Beyond Recall: A Dual-Process Cognitive Memory System for Self-Evolving LLM Agents". In: *arXiv preprint arXiv:2606.09483* (2026). DOI: 10.48550/arXiv.2606.09483.

[12] Santhosh Kumar Ravindran. "Portable Agent Memory: A Protocol for Cryptographically-Verified Memory Transfer Across Heterogeneous AI Agents". In: *arXiv preprint arXiv:2605.11032* (2026).

[13] Ciyan Ouyang and Rui Hou. "MemLineage: Lineage-Guided Enforcement for LLM Agent Memory". In: *arXiv preprint arXiv:2605.14421* (2026).

[14] Young Bin Park. "Graph-Native Cognitive Memory for AI Agents: Formal Belief Revision Semantics for Versioned Memory Architectures". In: *arXiv preprint arXiv:2603.17244* (2026).

[15] Ashish Gupta and Inderpal Singh Mumick. "Maintenance of Materialized Views: Problems, Techniques, and Applications". In: *IEEE Data Engineering Bulletin* 18.2 (1995), pp. 3–18.

[16] Ralph C. Merkle. "Protocols for Public Key Cryptosystems". In: *1980 IEEE Symposium on Security and Privacy*. 1980, pp. 122–134. DOI: 10.1109/SP.1980.10006.

[17] Carlos E. Alchourrón, Peter Gärdenfors, and David Makinson. "On the Logic of Theory Change: Partial Meet Contraction and Revision Functions". In: *The Journal of Symbolic Logic* 50.2 (1985), pp. 510–530. DOI: 10.2307/2274239.

[18] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. "A-MEM: Agentic Memory for LLM Agents". In: *Advances in Neural Information Processing Systems* 38 (2025). arXiv:2502.12110. DOI: 10.52202/085713-0593.

[19] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. "Memory OS of AI Agent". In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2025, pp. 25961–25970.

[20] Zhiyu Li et al. "MemOS: An Operating System for Memory-Augmented Generation (MAG) in Large Language Models". In: *arXiv preprint arXiv:2505.22101* (2025). DOI: 10.48550/arXiv.2505.22101.

[21] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. "Zep: A Temporal Knowledge Graph Architecture for Agent Memory". In: *arXiv preprint arXiv:2501.13956* (2025). DOI: 10.48550/arXiv.2501.13956.

[22] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval". In: *arXiv preprint arXiv:2401.18059* (2024).

[23] Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. "Generative Agents: Interactive Simulacra of Human Behavior". In: *arXiv preprint arXiv:2304.03442* (2023).

[24] Joon Sung Park et al. "LLM Agents Grounded in Self-Reports Enable General-Purpose Simulation of Individuals". In: *arXiv preprint arXiv:2411.10109* (2024).

[25] Alireza Salemi, Sheshera Mysore, Michael Bendersky, and Hamed Zamani. "LaMP: When Large Language Models Meet Personalization". In: *arXiv preprint arXiv:2304.11406* (2023).

[26] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory". In: *arXiv preprint arXiv:2410.10813* (2024).

[27] Haonan Bian et al. "RealMem: Benchmarking LLMs in Real-World Memory-Driven Interaction". In: *arXiv preprint arXiv:2601.06966* (2026).

[28] Timothy Lebo, Satya Sahoo, Deborah McGuinness, et al. *PROV-O: The PROV Ontology*. W3C Recommendation. World Wide Web Consortium, Apr. 2013. URL: https://www.w3.org/TR/prov-o/.

[29] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. "Provenance Semirings". In: *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. ACM, 2007, pp. 31–40. DOI: 10.1145/1265530.1265535.

[30] Elzo Brito dos Santos Filho. "ESAA: Event Sourcing for Autonomous Agents in LLM-Based Software Engineering". In: *arXiv preprint arXiv:2602.23193* (2026). DOI: 10.48550/arXiv.2602.23193.

[31] Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, et al. "Code as Agent Harness". In: *arXiv preprint arXiv:2605.18747* (2026).